CERTIFICACIÓN DATA ENGINEER · MÓDULO 2 · PYTHON II

Clase 2
Resumen de la clase

Condicionales, bucles, listas, textos y funciones: todo lo que vimos, con ejemplos para volver a probar en VS Code.

QUÉ VIMOS

1 Condicionales
`if`, `elif`, `else`, `and` y `or`: el programa toma decisiones.

2 Bucle for
Repetir algo por cada elemento de una lista o de un `range`.

3 Bucle while
Repetir mientras se cumpla una condición. `break` y `continue`.

4 Métodos de listas
Agregar, quitar, buscar y contar elementos.

5 Métodos de strings
Limpiar, separar y unir textos.

6 Funciones
Crear tus propias herramientas con `def` y `return`.

7 Mini quiz y errores
Las preguntas de la clase, con sus respuestas.

8 Práctica
Los 10 ejercicios de la clase, con pistas. Las soluciones están al final.

CÓMO USAR ESTE RESUMEN

- Lee la idea de cada tema y mira el ejemplo.
- Copia el ejemplo en VS Code, ejecútalo con ▶ y compara con lo que dice **Imprime**.
- Cambia los datos y vuelve a ejecutar: así se entiende qué hace cada línea.
- En los ejercicios, intenta primero sin ayuda. Si te trabas, mira las pistas; las soluciones están al final.

LA REGLA DE ORO: LOS DOS PUNTOS Y LA SANGRÍA

Las líneas que terminan en dos puntos (`if`, `elif`, `else`, `for`, `while`, `def`) abren un bloque. Todo lo que va adentro del bloque lleva **4 espacios** al principio: la sangría. Si falta, Python da `IndentationError`.

```
if nota >= 6:
    # Adentro del if: 4 espacios
    print("Aprobado")
# Afuera del if: se ejecuta siempre
print("Fin")
```

1 Condicionales: if, elif y else

> Sirven para que Python **tome decisiones**: ejecuta un bloque de código solo si se cumple una condición.
>
> **Como en la vida real:** como elegir qué ponerse. Si hace calor, remera; si no, si está templado, buzo; y si no, campera.

if y else: dos caminos

EJEMPLO

```
edad = 20

if edad >= 18:
    print("Mayor de edad")
else:
    print("Menor de edad")
```

IMPRIME

```
Mayor de edad
```

elif: más de dos caminos

EJEMPLO

```
nota = 10

if nota >= 9:
    print("Excelente")
elif nota >= 6:
    print("Aprobado")
else:
    print("Reprobado")
```

IMPRIME

```
Excelente
```

CÓMO LO LEE PYTHON

- Revisa las condiciones **de arriba hacia abajo** y entra solo en la **primera** que se cumple. Las demás las salta.
- `else` no lleva condición: es lo que pasa cuando no se cumplió ninguna.
- El orden importa: si preguntas primero por `nota >= 6`, un 10 imprime Aprobado.

Comparaciones

Operador	Pregunta	Ejemplo	Con nota = 7
`==`	¿Es igual?	`nota == 7`	`True`
`!=`	¿Es distinto?	`nota != 7`	`False`
`>` `<`	¿Es mayor? ¿Es menor?	`nota > 9`	`False`
`>=` `<=`	¿Es mayor o igual? ¿Es menor o igual?	`nota >= 6`	`True`

Ojo: un solo `=` guarda un valor (`nota = 7`). Dos, `==`, comparan (`nota == 7`).

Varias condiciones juntas: and, or y not

Palabra	Qué significa	Ejemplo	Con edad = 30 y es_feriado = False
`and`	Y: se tienen que cumplir **todas** las condiciones.	`edad >= 18 and edad < 65`	`True`
`or`	O: alcanza con que se cumpla **una**.	`edad < 12 or edad > 65`	`False`
`not`	Invierte: lo verdadero pasa a falso y al revés.	`not es_feriado`	`True`

EJEMPLO

```
tiene_entrada = True
tiene_documento = False

if tiene_entrada and tiene_documento:
    print("Puede entrar")
else:
    print("No puede entrar")
```

IMPRIME

```
No puede entrar
```

Con `and` hacen falta las dos cosas: tiene entrada, pero no documento. Si cambias `and` por `or`, alcanza con una e imprime Puede entrar. `True` (verdadero) y `False` (falso) se usan directo en el `if`, sin escribir `== True`.

PARA RECORDAR

- Después de `if`, `elif` y `else` van dos puntos `:`, y lo de adentro lleva sangría.
- Python entra solo en el primer bloque cuya condición se cumple.

2 Bucle for

`for` repite un bloque de código **por cada elemento** de una secuencia: una lista o un rango de números (`range`).

Como en la vida real: como subir una escalera. En cada escalón haces lo mismo, pero el número del escalón cambia.

Recorrer una lista

EJEMPLO

```
frutas = ["manzana", "pera", "naranja"]

for fruta in frutas:
    print(fruta)
```

IMPRIME

```
manzana
pera
naranja
```

En cada vuelta, `fruta` toma el siguiente elemento de la lista: primero manzana, después pera y al final naranja. Lo que tiene sangría debajo del `for` se repite en cada vuelta.

Repetir con range

EJEMPLO

```
for i in range(5):
    print(i)
```

IMPRIME

```
0
1
2
3
4
```

Escribes	Genera	Para qué
`range(5)`	0, 1, 2, 3, 4	Empieza en 0 y da 5 números.
`range(1, 4)`	1, 2, 3	Empieza en 1. El 4 no se incluye.
`range(1, 11)`	1, 2, ..., 10	Para contar del 1 al 10.

La regla: el último número de `range` **nunca se incluye**. Para contar del 1 al 10 se escribe `range(1, 11)`.

Sumar con un acumulador

Para sumar los elementos de una lista se usa un **acumulador**: una variable que empieza en 0 y a la que se le suma algo en cada vuelta. Es como una alcancía: empieza vacía y cada día le agregas lo que juntaste.

EJEMPLO

```
goles = [2, 0, 3]
total = 0

for g in goles:
    total = total + g

print("Goles en total:", total)
```

IMPRIME

```
Goles en total: 5
```

- `total = 0` es la alcancía vacía: va **antes** del `for`.
- `total = total + g` suma cada valor: 0 + 2 = 2, después 2 + 0 = 2 y después 2 + 3 = 5. También se escribe `total += g`.
- El `print` va **afuera** del `for` (sin sangría): se ejecuta una sola vez, al final.
- Un **contador** es lo mismo, pero sumando 1: `cantidad = cantidad + 1`.

PARA RECORDAR

- Lo que tiene sangría debajo del `for` se repite en cada vuelta.
- `range(inicio, fin)` llega hasta fin - 1.
- El acumulador se crea en 0 antes del `for` y se suma adentro.

3 Bucle while

`while` significa “mientras”: repite un bloque **mientras la condición sea verdadera**.

Como en la vida real: mientras quede agua en el vaso, tomas un trago. Cada trago deja menos agua; si nunca tomaras, el “mientras” no terminaría.

EJEMPLO

```
contador = 1

while contador <= 3:
    print("Vuelta", contador)
    contador += 1

print("Listo")
```

IMPRIME

```
Vuelta 1
Vuelta 2
Vuelta 3
Listo
```

`contador += 1` es lo mismo que `contador = contador + 1`. Cuando `contador` llega a 4, la condición es falsa y el programa sigue después del bucle.

> **Cuidado con el bucle infinito.** Algo adentro del bucle tiene que cambiar para que la condición alguna vez sea falsa. Si no, el programa no termina nunca. En VS Code se corta con **Ctrl + C** en la terminal.

break y continue

Palabra	Qué hace
`break`	Termina el bucle en ese momento.
`continue`	Salta el resto de esta vuelta y pasa a la siguiente.

EJEMPLO CON BREAK

```
while True:
    numero = int(input("Ingresa un número: "))
    if numero < 0:
        break          # Un negativo termina el bucle
    print(numero)
```

EN LA TERMINAL, SI ESCRIBES 4, 7 Y -1

```
Ingresa un número: 4
4
Ingresa un número: 7
7
Ingresa un número: -1
```

`while True` repetiría para siempre: la única salida es el `break`. `input()` lee lo que escribe el usuario (como texto) e `int()` lo convierte en número.

EJEMPLO CON CONTINUE

```
numero = 0

while numero < 5:
    numero += 1
    if numero % 2 == 0:
        continue       # Si es par, salta el print
    print(numero)
```

IMPRIME

```
1
3
5
```

`%` da el resto de una división: `numero % 2 == 0` pregunta si el número es par. Con los pares, `continue` salta el `print`. Por eso solo aparecen 1, 3 y 5.

¿for o while?

Usa `for` para recorrer una lista o para repetir una cantidad de veces que ya conoces. Usa `while` para repetir hasta que pase algo.

PARA RECORDAR

- `while` repite mientras la condición sea verdadera.
- Siempre tiene que cambiar algo adentro del bucle. Si no, es infinito.

4 Métodos de listas

Una lista guarda varios datos juntos, entre corchetes. Cada elemento tiene una **posición**, y la primera es la **0**. Los **métodos** son herramientas propias de la lista: se usan con un punto, como `lista.append(x)`.

Como en la vida real: como la lista del súper en el celular. Agregas cosas, borras lo que ya no hace falta y te fijas si algo ya está anotado.

EJEMPLO

```
compras = ["leche", "arroz", "café"]

print(compras[0])          # La primera posición es la 0
print(len(compras))        # Cantidad de elementos
print("arroz" in compras)  # ¿Está en la lista?
```

IMPRIME

```
leche
3
True
```

Los métodos que vimos

Cada fila parte de la misma lista: `lista = [1, 2, 3, 4, 5, 6, 7]`.

Escribes	Qué hace	Resultado
AGREGAR		
`lista.append(1000)`	Agrega un elemento al final.	Queda `[1, 2, 3, 4, 5, 6, 7, 1000]`
`lista.insert(1, 45)`	Agrega un elemento en una posición (acá, la 1).	Queda `[1, 45, 2, 3, 4, 5, 6, 7]`
`lista.extend([45, 76])`	Agrega varios elementos al final.	Queda `[1, 2, 3, 4, 5, 6, 7, 45, 76]`
QUITAR		
`lista.remove(3)`	Quita un valor (el primero que encuentra).	Queda `[1, 2, 4, 5, 6, 7]`
`lista.pop()`	Quita el último y lo devuelve.	Devuelve `7` y queda `[1, 2, 3, 4, 5, 6]`
`lista.pop(1)`	Quita el de una posición y lo devuelve.	Devuelve `2` y queda `[1, 3, 4, 5, 6, 7]`
BUSCAR Y CONTAR		
`lista.index(2)`	Dice en qué posición está un valor.	`1`
`lista.count(3)`	Cuenta cuántas veces aparece un valor.	`1`
`len(lista)`	Cuenta los elementos.	`7`
`3 in lista`	Pregunta si el valor está.	`True`

`3 not in lista`	Pregunta si el valor no está.	`False`

`remove` e `index` dan **ValueError** si el valor no está en la lista, como `lista.index(20)`. Pregunta antes con `in`:

```
if 20 in lista:
    lista.remove(20)
```

PARA RECORDAR

- Los métodos se usan con un punto: `lista.append(x)`.
- `append`, `insert`, `extend`, `remove` y `pop` cambian la lista original.
- Las posiciones empiezan en 0: `lista[0]` es el primer elemento.

5 Métodos de strings (textos)

Un string es un texto entre comillas. También tiene métodos, pero con una diferencia importante: un string es **inmutable**. Sus métodos no cambian el texto original: devuelven uno nuevo, y por eso hay que guardar el resultado.

Como en la vida real: como pasar un borrador en limpio. El borrador queda igual y te quedas con una copia prolija.

EJEMPLO

```
nombre = "  ANA  "
limpio = nombre.strip().lower()

print(limpio)
print(len(nombre), len(limpio))
```

IMPRIME

```
ana
7 3
```

`nombre` no cambió: sigue teniendo 7 caracteres. El texto limpio quedó en `limpio`. Los métodos se pueden encadenar: primero `strip()` y después `lower()`.

Limpiar y separar

Cada fila parte de `texto = "  Juan, 25, Buenos Aires  "`. Los resultados van entre comillas para que se vean los espacios.

Escribes	Qué hace	Resultado
LIMPIAR		
`texto.strip()`	Quita los espacios del principio y del final.	`'Juan, 25, Buenos Aires'`
`texto.lstrip()`	Quita los espacios del principio.	`'Juan, 25, Buenos Aires  '`
`texto.rstrip()`	Quita los espacios del final.	`'  Juan, 25, Buenos Aires'`
`texto.lower()`	Pasa todo a minúsculas.	`'  juan, 25, buenos aires  '`
`texto.upper()`	Pasa todo a mayúsculas.	`'  JUAN, 25, BUENOS AIRES  '`

`len(texto)`	Cuenta los caracteres, espacios incluidos.	`26`
SEPARAR		
`texto.split()`	Corta en los espacios y devuelve una lista.	`['Juan,', '25,', 'Buenos', 'Aires']`
`texto.split(",")`	Corta en cada coma.	`['  Juan', ' 25', ' Buenos Aires  ']`
`texto.strip().split(", ")`	Primero limpia y después corta en cada coma con espacio.	`['Juan', '25', 'Buenos Aires']`

Preguntas que responden True o False

Método	Pregunta	Ejemplos
`isdigit()`	¿Son todos dígitos?	`"25".isdigit()` da `True` `"2a".isdigit()` da `False`
`isalpha()`	¿Son todas letras?	`"Ana".isalpha()` da `True` `"Ana 25".isalpha()` da `False`
`islower()`	¿Está todo en minúsculas?	`"hola".islower()` da `True` `"Hola".islower()` da `False`

join: de lista a texto

`join` hace lo contrario de `split`: une los elementos de una lista en un solo texto. Antes del punto va lo que se pone entre las partes.

Escribes	Resultado
`"-".join(["rojo", "verde", "azul"])`	`'rojo-verde-azul'`
`"".join(["user", "@g", "mail.com"])`	`'user@gmail.com'`

Limpiar cada parte

EJEMPLO

```
texto = "  Juan, 25, Buenos Aires  "

partes = []
for item in texto.split(","):
    partes.append(item.strip())

print(partes)
```

IMPRIME

```
['Juan', '25', 'Buenos Aires']
```

Forma corta, la de la lámina: `[item.strip() for item in texto.split(",")]` hace lo mismo en una sola línea.

Ejemplo de la clase: corregir un email

EJEMPLO

```
mail = "  USER@MAIL.COM  "

mail = mail.strip()      # 'USER@MAIL.COM'
mail = mail.lower()      # 'user@mail.com'
mail = mail.split("@")   # ['user', 'mail.com']
mail.insert(1, "@g")     # ['user', '@g', 'mail.com']
mail = "".join(mail)     # 'user@gmail.com'

print(mail)
```

IMPRIME

```
user@gmail.com
```

Cada comentario muestra cuánto vale `mail` después de esa línea. Fíjate en la línea de `insert`: no lleva `mail =`. En ese momento `mail` es una lista, y los métodos de listas cambian la lista original. Los de strings, en cambio, devuelven un texto nuevo que hay que guardar.

PARA RECORDAR

- Los métodos de strings devuelven un texto nuevo: guárdalo, como en `mail = mail.strip()`.
- `split` pasa de texto a lista; `join` pasa de lista a texto.

6 Funciones

Una función es un bloque de código **con nombre** que se puede usar muchas veces. Recibe datos de entrada (**parámetros**), hace una tarea y puede **devolver** un resultado con `return`. Sirven para ordenar el código, reutilizarlo y entenderlo mejor.

Como en la vida real: una licuadora. Le pones ingredientes (parámetros), aprietas el botón (la llamas) y te devuelve un licuado (return) que puedes tomar o guardar.

Crear y llamar una función

EJEMPLO

```
def saludar(nombre):
    print("Hola", nombre)

saludar("Ana")
saludar("Luis")
```

IMPRIME

```
Hola Ana
Hola Luis
```

`def` crea la función y `nombre` es su parámetro. Crearla no ejecuta nada: el código de adentro corre recién cuando la llamas, como en `saludar("Ana")`.

return: devolver un resultado

EJEMPLO

```
def sumar(a, b):
    return a + b

resultado = sumar(3, 5)
print(resultado)
print(sumar(10, 20) * 2)
```

IMPRIME

```
8
60
```

`return` entrega el resultado: se puede guardar en una variable o seguir usándolo en una cuenta, como en `sumar(10, 20) * 2`.

¿print o return?

EJEMPLO

```
def doble_con_print(n):
    print(n * 2)        # Solo lo muestra

def doble_con_return(n):
    return n * 2        # Lo devuelve

a = doble_con_print(4)
b = doble_con_return(4)
print(a, b)
```

IMPRIME

```
8
None 8
```

La primera función muestra 8, pero no devuelve nada: `a` queda en `None` (“nada”). Si otra parte del programa va a usar el resultado, la función tiene que usar `return`.

Ejemplos de la clase

LOS PRIMEROS N NÚMEROS PARES

EJEMPLO

```
def n_pares(N):
    n = 0          # Cuántos pares encontramos
    idx = 1        # El número que estamos mirando
    while n < N:
        if idx % 2 == 0:
            n += 1
            print(idx)
        idx += 1

n_pares(5)
```

IMPRIME

```
2
4
6
8
10
```

`n` cuenta cuántos pares encontramos e `idx` es el número que estamos mirando. La versión general de la lámina, `n_kares(N, k)`, cambia `idx % 2` por `idx % k`: `n_kares(3, 4)` imprime 4, 8 y 12, los primeros 3 múltiplos de 4.

PRODUCTO EUCLÍDEO O LISTA MÁS CORTA

Una función que recibe dos listas de números. Si tienen la misma cantidad de elementos, calcula el producto euclídeo: multiplica posición por posición y suma todo. Si no, devuelve la lista más corta.

EJEMPLO

```python
def func_ejemplo(lista1, lista2):
    if len(lista1) == len(lista2):
        prod_euc = 0
        for idx in range(len(lista1)):
            prod_euc += lista1[idx] * lista2[idx]
        return f"El producto euclídeo es {prod_euc}"
    elif len(lista1) > len(lista2):
        return lista2
    else:
        return lista1

print(func_ejemplo([1, 2, 3], [6, 7, 8]))
print(func_ejemplo([1, 2, 3], [6, 7, 8, 5]))
```

IMPRIME

```
El producto euclídeo es 44
[1, 2, 3]
```

- Con [1, 2, 3] y [6, 7, 8]: 1 × 6 + 2 × 7 + 3 × 8 = 6 + 14 + 24 = 44.
- `range(len(lista1))` da las posiciones 0, 1 y 2, para recorrer las dos listas a la vez.
- `f"... {prod_euc}"` es un f-string: mete el valor de la variable dentro del texto.
- Una función puede tener varios `return`: termina en el primero que se ejecuta.

PARA RECORDAR

- `def` crea la función; para usarla hay que llamarla: `sumar(3, 5)`.
- `return` devuelve el resultado; `print` solo lo muestra.

7 Mini quiz y errores

¿Qué imprime?

PREGUNTA 1

```python
for i in range(3):
    print(i + 1)
```

OPCIONES

A	B ✓	C
0 1 2	1 2 3	1 2

Respuesta: B. `range(3)` da 0, 1 y 2, y a cada uno se le suma 1.

PREGUNTA 2

```
numeros = [1, 2, 3, 4, 5]

for n in numeros:
    if n > 2 and n < 5:
        print(n)
```

OPCIONES

A	B ✓	C
2 3 4	3 4	1 2 5

Respuesta: B. Con `and` se tienen que cumplir las dos condiciones: mayor que 2 **y** menor que 5. Solo 3 y 4 cumplen.

PREGUNTA 3

```
edad = 65

if edad < 18 or edad > 60:
    print("Descuento")
else:
    print("Sin descuento")
```

OPCIONES

A	B ✓	C
Sin descuento	Descuento	Error

Respuesta: B. Con `or` alcanza con una condición: 65 es mayor que 60.

Encuentra el error

Cada código tiene un error. Primero intenta encontrarlo; después mira la explicación.

CASO A

```
edad = 20

if edad >= 18:
print("Mayor de edad")
```

PYTHON DICE

```
IndentationError: expected an indented block after 'if' statement on line 3
```

QUÉ PASA

El `print` tiene que ir **adentro** del `if`, con 4 espacios de sangría.

ARREGLADO

```
edad = 20

if edad >= 18:
    print("Mayor de edad")
```

Imprime: Mayor de edad

CASO B

```
edad = 15

if edad > 18:
    print("Mayor")
else edad == 15:
    print("Quince")
```

PYTHON DICE

```
SyntaxError: expected ':'
```

QUÉ PASA

`else` no lleva condición: Python espera los dos puntos justo después. Para hacer otra pregunta se usa `elif`.

ARREGLADO

```
edad = 15

if edad > 18:
    print("Mayor")
elif edad == 15:
    print("Quince")
```

Imprime: Quince

CASO C

```
x = 10

if x = 5:
    print("Es cinco")
```

PYTHON DICE

```
SyntaxError: invalid syntax.
Maybe you meant '==' or ':='
instead of '='?
```

QUÉ PASA

Un solo `=` guarda un valor. Para comparar se usan dos: `==`.

ARREGLADO

```
x = 10

if x == 5:
    print("Es cinco")
```

Imprime: (nada: x vale 10, no 5)

CASO D

```
numeros = [1, 2, 3]
print(numeros[3])
```

PYTHON DICE

```
IndexError: list index out
of range
```

QUÉ PASA

La lista tiene 3 elementos y sus posiciones son 0, 1 y 2: la posición 3 no existe.

ARREGLADO

```
numeros = [1, 2, 3]
# La última posición es la 2
print(numeros[2])
```

Imprime: 3

CASO E

```
i = 0

while i < 5:
    print(i)
```

PYTHON DICE

```
(nada: el programa no
termina)
```

QUÉ PASA

No aparece ningún error, pero el programa **no termina nunca**: imprime 0 una y otra vez. `i` nunca cambia, así que `i < 5` siempre es verdadero.

ARREGLADO

```
i = 0

while i < 5:
    print(i)
    # Ahora i avanza
    i += 1
```

Imprime: 0, 1, 2, 3, 4

CASO F

```
edad = "20"

if edad > 18:
    print("Mayor")
```

PYTHON DICE

```
TypeError: '>' not supported
between instances of 'str'
and 'int'
```

QUÉ PASA

Con comillas, `"20"` es un texto, y Python no puede comparar un texto con un número.

ARREGLADO

```
edad = "20"

# int() lo pasa a número
if int(edad) > 18:
    print("Mayor")
```

Imprime: Mayor

Cómo leer un error

Cuando algo falla, Python muestra un mensaje en la terminal. Por ejemplo, el caso D da:

```
Traceback (most recent call last):
  File "ejercicio.py", line 2, in <module>
    print(numeros[3])
          ~~~~~~~^^^
IndexError: list index out of range
```

- `line 2` dice en qué línea está el problema.
- La **última línea** dice el tipo de error y qué pasó, en inglés: "índice de lista fuera de rango".

LOS ERRORES MÁS COMUNES

Error	Qué quiere decir
`SyntaxError`	Algo está mal escrito: suele faltar `:`, un paréntesis o una comilla, o hay un `=` donde va `==`.
`IndentationError`	Falta o sobra sangría: lo de adentro de `if`, `for`, `while` o `def` lleva 4 espacios.
`NameError`	Usas un nombre que no existe: un error de tipeo o una variable que todavía no creaste.
`TypeError`	Mezclas tipos que no combinan, como un texto y un número.
`IndexError`	Pides una posición que la lista no tiene.
`ValueError`	El valor no sirve: por ejemplo, `remove` o `index` de algo que no está en la lista, o `int("hola")`.
El programa no termina	Casi siempre es un `while` cuya condición nunca se vuelve falsa. Córtalo con **Ctrl + C** en la terminal.

Palabras clave de la clase

Palabra	Qué significa
Variable	Un nombre que guarda un dato: `nota = 9.5`.
Condición	Una pregunta que da verdadero o falso: `nota >= 6`.
True / False	Verdadero y falso: los dos resultados posibles de una condición.
Bloque y sangría	Las líneas que van adentro de un `if`, `for`, `while` o `def` llevan 4 espacios al principio.
Bucle	Código que se repite: `for` (por cada elemento) y `while` (mientras se cumpla algo).
Acumulador	Una variable que empieza en 0 y a la que se le suma algo en cada vuelta.
Lista	Varios datos juntos, entre corchetes: `[200, 350, 100]`.
Posición (índice)	El lugar de un elemento en una lista. La primera posición es la 0: `lista[0]`.
String	Un texto, entre comillas: `"Hola"`. Es inmutable: sus métodos devuelven un texto nuevo.
Método	Una herramienta propia de un dato, que se usa con un punto: `lista.append(x)`, `texto.lower()`.
Función	Un bloque de código con nombre que se puede usar muchas veces. Se crea con `def`.
Parámetro	El dato que recibe una función, entre paréntesis: en `def doble(numero)`, el parámetro es `numero`.
return	Devuelve el resultado de una función para seguir usándolo. `print` solo lo muestra.

8 Práctica: los 10 ejercicios de la clase

Estos son los 10 ejercicios que resolvimos en clase. Para practicar en casa:

1. Crea un archivo nuevo en VS Code para cada ejercicio, por ejemplo `ejercicio1.py`.
2. Copia los datos y escribe tu código debajo.
3. Ejecútalo con ▶ y compara con **Debe imprimir**.
4. Cambia los datos por los de **Pruébalo también con**: un buen programa funciona con cualquier dato.
5. Si te trabas, lee las pistas de a una. Las soluciones están al final.

EJERCICIO 1 · IF / ELIF / ELSE

¿Cómo le fue en el examen?

Tenemos la nota de un examen, de 0 a 10, en la variable `nota`. El programa tiene que decir cómo le fue al alumno.

QUÉ TIENE QUE HACER EL PROGRAMA

1. Si la nota es 9 o más, imprime `Excelente`.
2. Si no, pero es 6 o más, imprime `Aprobado`.
3. En cualquier otro caso, imprime `Reprobado`.

DATOS

```
nota = 9.5
```

DEBE IMPRIMIR

```
Excelente
```

PRUÉBALO TAMBIÉN CON

Datos	Debe imprimir
`nota = 9`	`Excelente`
`nota = 6`	`Aprobado`
`nota = 5.9`	`Reprobado`

SI TE TRABAS: PISTAS (LÉELAS DE A UNA)

1. Hay tres respuestas posibles, así que vas a usar `if`, `elif` y `else`.
2. Pregunta primero por la nota más alta (`nota >= 9`) y después por `nota >= 6`. Si lo haces al revés, un 10 diría Aprobado.
3. Cada línea con `if`, `elif` o `else` termina con dos puntos `:` y el `print` va debajo, con 4 espacios de sangría.

Desafío extra: Si la nota es menor que 0 o mayor que 10, imprime `Nota inválida`.

EJERCICIO 2 · AND / OR

¿Tiene descuento en el cine?

Un cine tiene precios con descuento. Con los datos `edad` y `es_miercoles` (True o False), el programa tiene que decir qué precio paga la persona.

QUÉ TIENE QUE HACER EL PROGRAMA

1. Hay descuento si la persona tiene menos de 12 años.
2. También hay descuento si tiene más de 65 años.
3. Y también si es miércoles: ese día hay descuento para todos.
4. Si se cumple alguna de las tres, imprime `Con descuento`. Si no, imprime `Precio normal`.

DATOS

```
edad = 8
es_miercoles = False
```

DEBE IMPRIMIR

```
Con descuento
```

PRUÉBALO TAMBIÉN CON

Datos	Debe imprimir
`edad = 30`, `es_miercoles = True`	`Con descuento`
`edad = 70`, `es_miercoles = False`	`Con descuento`
`edad = 30`, `es_miercoles = False`	`Precio normal`
`edad = 12`, `es_miercoles = False`	`Precio normal`

SI TE TRABAS: PISTAS (LÉELAS DE A UNA)

1. Hay tres motivos para tener descuento y alcanza con uno. ¿Qué palabra se usa cuando alcanza con una condición: `and` u `or`?
2. Los tres motivos son `edad < 12`, `edad > 65` y `es_miercoles`. Únelos con `or` en un solo `if`.
3. Queda así: `if edad < 12 or edad > 65 or es_miercoles:` y debajo los dos `print` con su `else`.

Desafío extra: Cambia la regla: los miércoles hay descuento solo para mayores de 18. ¿Dónde agregarías un `and`?

EJERCICIO 3 · FOR + RANGE

La tabla de multiplicar

Con el dato `numero`, el programa tiene que mostrar su tabla de multiplicar del 1 al 10, un resultado por línea.

QUÉ TIENE QUE HACER EL PROGRAMA

1. Repite 10 veces, con un contador que vaya del 1 al 10.
2. En cada vuelta imprime la multiplicación con este formato: `7 x 1 = 7`.

DATOS

```
numero = 7
```

DEBE IMPRIMIR

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

PRUÉBALO TAMBIÉN CON

Datos	Debe imprimir

`numero = 3`	`3 x 1 = 3`
	`3 x 2 = 6`
	`...`
	`3 x 10 = 30`

SI TE TRABAS: PISTAS (LÉELAS DE A UNA)

1. Necesitas un `for` que dé 10 vueltas, con una variable que vaya del 1 al 10.
2. `range(1, 11)` genera los números del 1 al 10.
3. En cada vuelta: `print(numero, "x", i, "=", numero * i)`

Desafío extra: Imprime solo los resultados que son pares.

EJERCICIO 4 · FOR + ACUMULADOR

Sumar las ventas del día

La lista `ventas` tiene lo que se vendió en cada hora del día. El programa tiene que calcular el total vendido y el promedio.

QUÉ TIENE QUE HACER EL PROGRAMA

1. Suma todas las ventas con un `for` (sin usar `sum()`).
2. Imprime el total así: `Total: 1250`.
3. Imprime el promedio (el total dividido la cantidad de ventas) así: `Promedio: 250.0`.

DATOS

```
ventas = [200, 350, 100, 400, 200]
```

DEBE IMPRIMIR

```
Total: 1250
Promedio: 250.0
```

PRUÉBALO TAMBIÉN CON

Datos	Debe imprimir
`ventas = [10, 20]`	`Total: 30` `Promedio: 15.0`
`ventas = [99]`	`Total: 99` `Promedio: 99.0`

SI TE TRABAS: PISTAS (LÉELAS DE A UNA)

1. Antes del `for`, crea la alcancía: `total = 0`.
2. Dentro del `for`, suma cada venta: `total = total + venta`.
3. La cantidad de ventas la da `len(ventas)`. Imprime con `print("Total:", total)` y `print("Promedio:", total / len(ventas))`.

Desafío extra: Si la lista está vacía, el promedio da error. Usa un `if` para imprimir `Sin ventas` en ese caso.

EJERCICIO 5 · WHILE

Leer datos hasta el -1

Un sensor manda la lista `lecturas` y marca el final de los datos con un `-1`. Lo que viene después del -1 no se usa.

QUÉ TIENE QUE HACER EL PROGRAMA

1. Recorre la lista con un `while`, de a una posición.
2. Suma cada lectura hasta encontrar el `-1`. El -1 no se suma.
3. Imprime cuántas lecturas sumaste: `Lecturas: 3`.
4. Imprime la suma: `Suma: 36`.

DATOS

```
lecturas = [12, 15, 9, -1, 20, 7]
```

DEBE IMPRIMIR

```
Lecturas: 3
Suma: 36
```

PRUÉBALO TAMBIÉN CON

Datos	Debe imprimir
`lecturas = [5, -1]`	`Lecturas: 1` `Suma: 5`
`lecturas = [-1, 4, 4]`	`Lecturas: 0` `Suma: 0`

SI TE TRABAS: PISTAS (LÉELAS DE A UNA)

1. Usa una variable `i = 0` para ir posición por posición: `lecturas[0]` es 12, `lecturas[1]` es 15, y así.
2. La condición es "mientras la lectura actual no sea -1": `while lecturas[i] != -1:`
3. Dentro del bucle suma `lecturas[i]` a una variable `suma` y avanza con `i = i + 1`. Al terminar, `i` es la cantidad de lecturas.

Desafío extra: ¿Y si la lista no trae `-1`? Agrega `i < len(lecturas) and` al principio de la condición para que no falle.

EJERCICIO 6 · MÉTODOS DE LISTAS

La lista del súper

Tenemos la lista del súper en la variable `compras` y hay que actualizarla.

QUÉ TIENE QUE HACER EL PROGRAMA

1. Agrega `"pan"` al final de la lista.
2. Si está `"leche"`, quítala.
3. Si no está `"huevos"`, agrégalo.
4. Imprime la lista y, en otra línea, la cantidad de productos así: `Total: 4 productos`.

DATOS

```
compras = ["leche", "arroz", "café"]
```

DEBE IMPRIMIR

```
['arroz', 'café', 'pan', 'huevos']
Total: 4 productos
```

PRUÉBALO TAMBIÉN CON

Datos	Debe imprimir
`compras = ["huevos", "leche"]`	`['huevos', 'pan']` `Total: 2 productos`
`compras = ["arroz"]`	`['arroz', 'pan', 'huevos']` `Total: 3 productos`

SI TE TRABAS: PISTAS (LÉELAS DE A UNA)

1. Haz los pasos en orden, uno debajo del otro: `append`, después `remove` y después `append` otra vez.
2. `remove` da error si el elemento no está. Por eso, pregunta antes: `if "leche" in compras:`
3. Para "si no está" se usa `not in`: `if "huevos" not in compras:`. La cantidad la da `len(compras)`.

Desafío extra: Antes de imprimir, ordena la lista alfabéticamente con `compras.sort()`.

EJERCICIO 7 · MÉTODOS DE STRINGS

Limpiar emails

La lista `emails` llegó de un formulario: algunos tienen espacios de más y mayúsculas.

QUÉ TIENE QUE HACER EL PROGRAMA

1. Crea una lista nueva y vacía llamada `limpios`.
2. Por cada email, quítale los espacios de los costados y pásalo a minúsculas.
3. Agrega cada email limpio a `limpios`.
4. Al final, imprime `limpios`.

DATOS

```
emails = ["  Ana@Mail.com", "LUIS@MAIL.COM  ", " marta@mail.com "]
```

DEBE IMPRIMIR

```
['ana@mail.com', 'luis@mail.com', 'marta@mail.com']
```

PRUÉBALO TAMBIÉN CON

Datos	Debe imprimir
`emails = ["X@Y.COM"]`	`['x@y.com']`
`emails = []`	`[]`

SI TE TRABAS: PISTAS (LÉELAS DE A UNA)

1. Empieza con una lista vacía, `limpios = []`, y recorre `emails` con un `for`.
2. Para cada email, `email.strip().lower()` te da la versión limpia.
3. Agrégala con `limpios.append(...)` y, después del `for` (sin sangría), escribe `print(limpios)`.

Desafío extra: Descarta los emails que no tengan `@` (pista: `"@" in email`).

EJERCICIO 8 · SPLIT / JOIN

Separar los datos de una línea

Cada línea de un archivo CSV trae `nombre,edad,ciudad` separados por comas, pero con espacios de más. Tenemos una de esas líneas en la variable `linea`.

QUÉ TIENE QUE HACER EL PROGRAMA

1. Corta la línea en cada coma y quítale los espacios a cada parte.
2. Imprime cada dato en su línea: `Nombre: Juan`, `Edad: 25` y `Ciudad: Buenos Aires`.
3. Imprime la fila limpia, unida con punto y coma: `Limpia: Juan;25;Buenos Aires`.

DATOS

```
linea = "  Juan , 25,Buenos Aires "
```

DEBE IMPRIMIR

```
Nombre: Juan
Edad: 25
Ciudad: Buenos Aires
Limpia: Juan;25;Buenos Aires
```

PRUÉBALO TAMBIÉN CON

Datos	Debe imprimir

`linea = "ana,31,  Córdoba"`	`Nombre: ana`
	`Edad: 31`
	`Ciudad: Córdoba`
	`Limpia: ana;31;Córdoba`

SI TE TRABAS: PISTAS (LÉELAS DE A UNA)

1. `linea.split(",")` te da una lista con 3 partes, pero cada parte todavía tiene espacios.
2. Recorre las partes con un `for` y guarda cada `parte.strip()` en una lista nueva llamada `campos`.
3. `campos[0]` es el nombre, `campos[1]` la edad y `campos[2]` la ciudad. Para la última línea: `print("Limpia:", ";".join(campos))`.

Desafío extra: Convierte la edad a número con `int(campos[1])` e imprime qué edad tendrá la persona el año próximo.

EJERCICIO 9 · FUNCIONES Y RETURN

Mi primera función

Vamos a crear nuestra primera función. Las últimas líneas del código para empezar ya la prueban.

QUÉ TIENE QUE HACER EL PROGRAMA

1. Completa la función `n_pares(n)`.
2. Tiene que **devolver** (con `return`, no con `print`) una lista con los primeros `n` números pares.
3. Por ejemplo, `n_pares(5)` devuelve `[2, 4, 6, 8, 10]`.

PARA EMPEZAR, COPIA ESTO EN VS CODE

```
def n_pares(n):
    # Escribe aquí tu código
    pass   # pass no hace nada: bórralo cuando escribas tu código

# --- Probamos la función ---
print(n_pares(5))
```

DEBE IMPRIMIR

```
[2, 4, 6, 8, 10]
```

PRUÉBALO TAMBIÉN CON

Prueba	Debe imprimir
`print(n_pares(1))`	`[2]`
`print(n_pares(0))`	`[]`

SI TE TRABAS: PISTAS (LÉELAS DE A UNA)

1. Dentro de la función, crea una lista vacía: `pares = []`.
2. Con `for i in range(1, n + 1):` cada par es `i * 2`. Agrégalo con `pares.append(i * 2)`.
3. Al final, afuera del `for` pero adentro de la función, escribe `return pares`.

Desafío extra: Escribe `n_multiplos(n, k)`, que devuelva los primeros `n` múltiplos de `k`.

EJERCICIO 10 · TODO JUNTO

Todo junto: el boletín

Armamos un boletín con dos funciones que trabajan juntas. Las últimas líneas del código para empezar ya las prueban.

QUÉ TIENE QUE HACER EL PROGRAMA

1. Completa `clasificar(nota)`: tiene que devolver `"Excelente"`, `"Aprobado"` o `"Reprobado"`, con las reglas del ejercicio 1.
2. Completa `contar_aprobados(notas)`: tiene que usar `clasificar` y devolver cuántas notas aprueban.
3. Excelente también cuenta como aprobado.

PARA EMPEZAR, COPIA ESTO EN VS CODE

```
def clasificar(nota):
    # Reglas del ejercicio 1, pero con return en lugar de print
    pass


def contar_aprobados(notas):
    # Usa clasificar() dentro de un for
    pass

# --- Probamos la función ---
print(clasificar(9.5))
print(contar_aprobados([7, 4, 10, 6, 3, 9]))
```

DEBE IMPRIMIR

```
Excelente
4
```

PRUÉBALO TAMBIÉN CON

Prueba	Debe imprimir
`print(clasificar(4))`	`Reprobado`
`print(contar_aprobados([1, 2]))`	`0`
`print(contar_aprobados([]))`	`0`

SI TE TRABAS: PISTAS (LÉELAS DE A UNA)

1. Para `clasificar`, copia el if/elif/else del ejercicio 1 y cambia cada `print(...)` por `return ...`.
2. En `contar_aprobados`, empieza con `aprobados = 0` y recorre `notas` con un `for`.
3. Para cada nota: `if clasificar(nota) != "Reprobado":` suma 1. Al final, `return aprobados`.

Desafío extra: Agrega `promedio(notas)` y muestra el promedio del curso con un decimal usando `round(valor, 1)`.

✓ Soluciones propuestas

Hay muchas formas correctas de resolver cada ejercicio: esta es una. Los comentarios (en verde, después de `#`) explican cada paso.

1. ¿Cómo le fue en el examen? · if / elif / else

```
# Datos (puedes cambiarlos y volver a ejecutar)
nota = 9.5

# --- Tu código ---
if nota >= 9:            # ¿La nota es 9 o más?
    print("Excelente")
elif nota >= 6:          # Si no, ¿es 6 o más?
    print("Aprobado")
else:                    # Si no se cumplió ninguna de las anteriores
    print("Reprobado")
```

PARA RECORDAR

- Python revisa las condiciones de arriba hacia abajo y entra solo en la primera que se cumple.
- Después de `if`, `elif` y `else` van dos puntos, y lo de adentro va con sangría.

2. ¿Tiene descuento en el cine? · and / or

```
# Datos (puedes cambiarlos y volver a ejecutar)
edad = 8
es_miercoles = False

# --- Tu código ---
# Alcanza con que se cumpla UNA de las tres condiciones: usamos or
if edad < 12 or edad > 65 or es_miercoles:
    print("Con descuento")
else:                    # No se cumplió ninguna
    print("Precio normal")
```

PARA RECORDAR

- `and`: se tienen que cumplir todas. `or`: alcanza con una.
- Un dato que ya es True o False se usa directo en el `if`, sin escribir `== True`.

3. La tabla de multiplicar · for + range

```
# Datos (puedes cambiarlos y volver a ejecutar)
numero = 7

# --- Tu código ---
for i in range(1, 11):                  # i toma los valores 1, 2, ..., 10
    print(numero, "x", i, "=", numero * i)   # Se repite en cada vuelta
```

PARA RECORDAR

- `range(inicio, fin)` llega hasta fin - 1: para contar del 1 al 10 se escribe `range(1, 11)`.
- Lo que tiene sangría debajo del `for` se repite en cada vuelta.

4. Sumar las ventas del día · for + acumulador

```
# Datos (puedes cambiarlos y volver a ejecutar)
ventas = [200, 350, 100, 400, 200]

# --- Tu código ---
total = 0                    # La alcancía empieza vacía
for venta in ventas:         # Recorremos cada venta de la lista
    total = total + venta    # y la sumamos al total
print("Total:", total)                  # Afuera del for: se imprime una vez
print("Promedio:", total / len(ventas))   # len() cuenta las ventas
```

PARA RECORDAR

- El acumulador se crea en 0 antes del `for` y se suma adentro del `for`.
- `len(lista)` dice cuántos elementos tiene la lista.

5. Leer datos hasta el -1 · while

```
# Datos (puedes cambiarlos y volver a ejecutar)
lecturas = [12, 15, 9, -1, 20, 7]

# --- Tu código ---
i = 0       # Posición que estamos mirando
suma = 0    # Acumulador

while lecturas[i] != -1:        # Mientras no lleguemos al -1
    suma = suma + lecturas[i]   # Sumamos la lectura actual
    i = i + 1                   # Pasamos a la siguiente (sin esto no termina)

print("Lecturas:", i)           # i quedó en la cantidad de lecturas sumadas
print("Suma:", suma)
```

PARA RECORDAR

- `while` repite mientras la condición sea verdadera.
- Siempre tiene que cambiar algo dentro del bucle; si no, es infinito (en VS Code se corta con Ctrl + C).

6. La lista del súper · Métodos de listas

```
# Datos (puedes cambiarlos y volver a ejecutar)
compras = ["leche", "arroz", "café"]

# --- Tu código ---
compras.append("pan")             # 1) Agregamos pan al final

if "leche" in compras:            # 2) Solo si está la leche...
    compras.remove("leche")       #    ...la quitamos (si no, daría error)

if "huevos" not in compras:       # 3) Si NO están los huevos...
    compras.append("huevos")      #    ...los agregamos

print(compras)
print("Total:", len(compras), "productos")
```

PARA RECORDAR

- Los métodos se usan con un punto: `lista.append(x)`, `lista.remove(x)`.
- Antes de usar `remove`, pregunta con `in` si el elemento está.

7. Limpiar emails · Métodos de strings

```
# Datos (puedes cambiarlos y volver a ejecutar)
emails = ["  Ana@Mail.com", "LUIS@MAIL.COM  ", " marta@mail.com "]

# --- Tu código ---
limpios = []                                # Lista nueva, vacía
for email in emails:                        # Recorremos cada email
    limpios.append(email.strip().lower())   # Lo limpiamos y lo guardamos
print(limpios)                              # Afuera del for: una sola vez
```

PARA RECORDAR

- Los métodos de texto devuelven un texto nuevo: guarda el resultado en una variable o en una lista.
- Se pueden encadenar: `texto.strip().lower()`.

8. Separar los datos de una línea · split / join

```
# Datos (puedes cambiarlos y volver a ejecutar)
linea = "  Juan , 25,Buenos Aires "

# --- Tu código ---
campos = []                           # Acá guardamos los datos limpios
for parte in linea.split(","):        # Cortamos la línea en cada coma
    campos.append(parte.strip())      # Quitamos los espacios de cada parte

print("Nombre:", campos[0])           # Las posiciones empiezan en 0
print("Edad:", campos[1])
print("Ciudad:", campos[2])
print("Limpia:", ";".join(campos))    # Unimos con punto y coma
```

PARA RECORDAR

- `split` pasa de texto a lista; `join` pasa de lista a texto.
- En una lista, la primera posición es la 0.

9. Mi primera función · Funciones y return

```
def n_pares(n):
    pares = []                  # Lista vacía para ir guardando
    for i in range(1, n + 1):   # i va de 1 hasta n
        pares.append(i * 2)     # El par número i es i * 2
    return pares                # Devolvemos la lista completa

# --- Probamos la función ---
print(n_pares(5))
```

PARA RECORDAR

- `def` crea la función; para usarla hay que llamarla: `n_pares(5)`.
- `return` devuelve el resultado; `print` solo lo muestra.

10. Todo junto: el boletín · Todo junto

```
def clasificar(nota):
    if nota >= 9:               # Mismas reglas del ejercicio 1...
        return "Excelente"      # ...pero devolvemos en lugar de imprimir
    elif nota >= 6:
        return "Aprobado"
    else:
        return "Reprobado"


def contar_aprobados(notas):
    aprobados = 0                           # Contador
    for nota in notas:                      # Recorremos todas las notas
        if clasificar(nota) != "Reprobado":  # Usamos la otra función
            aprobados = aprobados + 1
    return aprobados

# --- Probamos la función ---
print(clasificar(9.5))
print(contar_aprobados([7, 4, 10, 6, 3, 9]))
```

PARA RECORDAR

- Una función puede usar a otra: así se divide un problema grande en partes.
- Si otra función va a usar el resultado, hay que devolverlo con `return`, no imprimirlo.